

GitHub入門ガイド

GitHubを通じて、Gitの概念・コマンド・
開発ワークフローを学ぼう

初学者向け



目次

- 01  GitHubとは何か？
- 02  GitHubの裏側にある「Git」とは
- 03  リポジトリ — コードの保管場所
- 04  ブランチ — 安全な「下書き」スペース
- 05  開発の基本ワークフロー
- 06  コマンドリファレンス
- 07  知っておきたいTips
- 08  用語集
- 09  実践課題

GitHubとは何か？

プログラムコードのための、共有・履歴管理・レビュー機能付きクラウドサービス



共同作業が簡単

複数人で同じプロジェクトを
安全に編集・管理できる



変更履歴が完全に残る

いつ・誰が・何を変えたか
記録されバグ特定が迅速に



世界中の開発者と繋がる

オープンソース開発の
中心地

Before vs After

× ファイルをメールで送り合い、最新版が不明

◎ すべての変更がWebで確認、常に最新版を管理

GitHubの裏側にある「Git」とは

Git

変更履歴を管理する「ツール（仕組み）」

カーエンジン（動力となる仕組み）

GitHub

Gitを活用した「ウェブサービス」

カーナビ付きの快適な車

Gitが記録していること

記録する情報	例
いつ 変更したか	2025-06-01 14:32
誰が 変更したか	tanaka@example.com
何を 変更したか	ログイン画面のボタンの色を変更
なぜ 変更したか	コミットメッセージとして記録

コミット (Commit) – 変更の「セーブポイント」

コミットとは、変更内容をGitに「この状態として記録する」操作です。

ゲームのセーブポイントをイメージするとわかりやすいです。

Terminal

ファイルを編集した後、コミットするまでの流れ

git add .

変更ファイルを「記録予定リスト」に追加

git commit -m "ログイン機能を追加"

メッセージをつけて変更を記録



"" の中に書くメッセージが、GitHub上で変更履歴として表示されます。

「何をしたか」が一目でわかるメッセージを書くのがマナーです。

リポジトリコードの保管場所

リポジトリ（Repository）とは、コードとその変更履歴をすべて格納する「保管場所」です。

1つのプロジェクト = 1つのリポジトリ

ローカルリポジトリ

場所: 自分のPC上

用途: 実際に作業する場所

リモートリポジトリ

場所: GitHub（サーバー）上

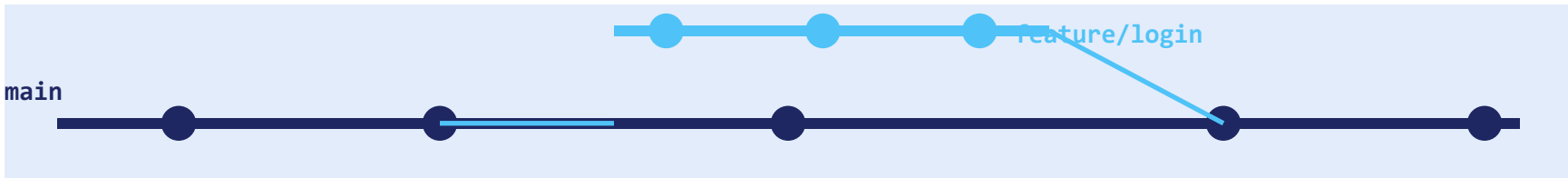
用途: チームとの共有・バックアップ

最初のステップ：クローン（clone）

```
git clone https://github.com/チーム名/リポジトリ名.git  
cd リポジトリ名
```

ブランチ – 安全な「下書き」スペース

ブランチ（Branch）とは、プロジェクトの作業履歴を「枝分かれさせる」機能です。完成版（main）を守りながら、安全に開発を進められます。



ローカルブランチ（自分のPC）で作業し、git push でリモートブランチ（GitHub上）に送信してチームと共有できます。

ブランチ名	役割
main	常に動く「完成版」。直接編集しない
feature/新機能名	新機能を開発するための「下書き」スペース
bugfix/バグ名	バグを修正するための作業スペース

```
git checkout -b feature/login
```

開発の基本ワークフロー

5つのステップで開発が進みます



① 分岐 → ② 作業 → ③ プッシュ → ④ レビュー (PR) → ⑤ 合流

ワークフロー STEP 1～3

1 ブランチを作る（分岐）

```
git checkout -b feature/新機能名
```

2 コードを編集してコミット（作業）

```
git status  
git add .  
git commit -m "ログイン画面のUIを修正"
```

ステージングとは？ git add は「このファイルの変更を次のコミットに含める」と宣言する操作です。

3 GitHubに送信する（プッシュ）

```
git push
```

初回: git push -u origin feature/新機能名

ワークフロー STEP 4～5

4 プルリクエスト（PR）を作成する

「この作業ブランチの変更を、mainに取り込んでください」と
チームに依頼する仕組み。レビュアーがコードを確認し、問題なければ承認します。

- 1 リポジトリページを開く
- 2 「Pull requests」 タブをクリック
- 3 「New pull request」 をクリック
- 4 自分の作業ブランチを選択
- 5 タイトル・説明を記入して作成

5 マージして完了（合流）

レビューが承認されたら「Merge pull request」をクリック → mainに合流！

```
git switch main && git pull && git branch -d feature/新機能名
```

コマンドリファレンス 早見表

コマンド	タイミング	意味・用途
<code>git clone [URL]</code>	最初の1回	リポジトリを手元にコピー
<code>git checkout -b [名前]</code>	作業開始前	新しい作業ブランチを作成して移動
<code>git status</code>	いつでも	変更されたファイルを確認
<code>git add .</code>	コミット前	すべての変更をステージング
<code>git commit -m "メモ"</code>	add後	変更を記録（セーブポイント作成）
<code>git push</code>	コミット後	GitHubに変更を送信
<code>git pull</code>	作業開始時	GitHubの最新状態を手元に反映
<code>git branch</code>	いつでも	ブランチ一覧と現在地を確認
<code>git switch [名前]</code>	ブランチ移動時	指定のブランチに移動
<code>git merge main</code>	定期的に	mainの最新変更を自分のブランチに取込
<code>git log</code>	いつでも	コミット履歴を確認
<code>git branch -d [名前]</code>	マージ後	不要なブランチを削除
<code>git revert [コミットID]</code>	コミット後	誤ったコミットを取り消す

知っておきたい Tips



常に最新の状態で作業する

他の人の変更が先にmainにマージされると、自分のブランチが古くなります。
定期的に以下のコマンドで最新状態を取り込みましょう。

```
git fetch origin  
git merge origin/main
```

コンフリクト予防のコツ：定期的な `git merge main` がポイント！



.gitignore を設定する

パスワードやAPIキーが入ったファイルをGitHubに上げると、世界中に公開される危険があります。

```
# .gitignore の記載例  
.env           # 環境変数ファイル  
node_modules/ # ライブラリフォルダ  
*.log          # ログファイル
```

用語集

用語	説明
Git	ファイルの変更履歴を管理するツール
GitHub	Gitを活用したウェブサービス
リポジトリ	コードと変更履歴の「保管場所」
ブランチ	作業を枝分かれさせる機能
main	本番の「完成版」ブランチ
コミット	変更内容を記録する操作（セーブポイント）
ステージング	コミットする変更を選ぶ中間ステップ (git add)
プッシュ (push)	手元の変更をGitHubに送信
プル (pull)	GitHubの最新状態を手元に反映
クローン (clone)	リモートリポジトリを手元にコピー
マージ (merge)	ブランチの変更を別のブランチに合流
プルリクエスト (PR)	変更をmainに取り込むようレビューを依頼
コンフリクト	同じ箇所を複数人が変更した際の「競合」
.gitignore	Git管理対象から除外するファイルを指定

実践課題



以下のURLから課題の確認をお願いします！

README.mdに課題説明が書かれているので、その説明に沿って課題に取り掛かって下さい

<https://github.com/yasuo-kouki/Hack-Playground>



Let's Start GitHub!

さあ、GitHubを使って開発を始めよう！
